DATA PRODUCT CASE STUDY

# PRIME: one governed data layer for a multi-tenant product

Then the same approach, live on City of Calgary transit data.

**Cody Chandler**
Data and Business Analyst interview · The City of Calgary

# How I worked, then the live product

**1**

## Problem

Q1

**2**

## What people needed

Q1

**3**

## Strategy and build

Q2 · Q3

**4**

## Results

Q2

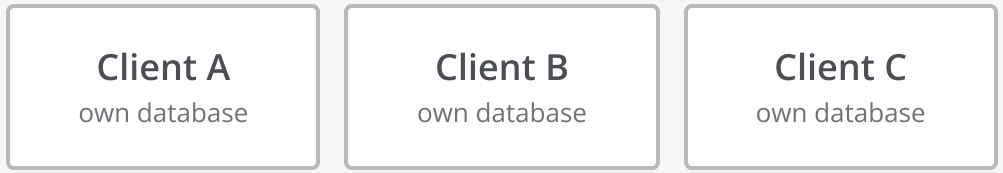
**5**

## Live demo

Q3 · Q5

Then the challenges (Q4) and what I'd build next (Q6).

# Multi-tenancy made accurate reporting hard




↓ manual weekly refresh ↓

**Same question, different answers**

## Stale

Data was up to 7 days old.

## Fragmented

Every cross-client question went through engineering.

## Unowned

No owner and no service levels when it broke.

# Three needs shaped every decision

## One version of the truth

CS, Product and clients see the same numbers.

→ One definition across every client

## Fresh and dependable

A known refresh schedule, and someone accountable.

→ Daily refresh and a data contract

## Self-serve

Routine questions answered without a ticket, by people or AI.

→ Documented tables, one grain each

# Fix the layer, not the reports

## 1 Consolidate once

One governed Snowflake layer instead of patching each client database.

## 2 Contract before code

Owners, refresh targets and incident tiers agreed first.

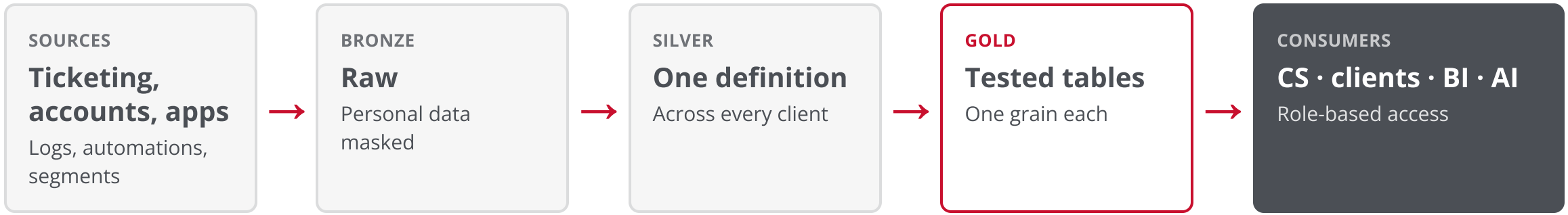
## 3 Model by grain

Each table answers one shape of question, so nothing double counts.

SEQUENCED SO EACH STEP WAS USABLE

1. Agree the contract
2. Land and protect the data
3. Define it once
4. Serve dashboards and data shares
5. Add AI, internal bot first

# PRIME: one layer every consumer reads from




Snowflake · dbt · 100+ tested models · Airflow · Freshness checked every 30 min · AI error digests

**My role:** Director of Product Innovation & Analytics. I owned PRIME from planning to general availability, working with Data Engineering and Customer Success.

# What changed

~~7 days~~

## < 12 hrs

Data freshness

## 100+

Tested models, replacing a database per client

## 30 min

To flag a missed refresh

## Self-serve

Routine Customer Success questions

**When the dashboard, the client export and the AI give the same number, people stop checking and start deciding.**

PRIME'S DATA BELONGS TO CLIENTS, SO I REBUILT THE APPROACH ON OPEN DATA

# Live on City of Calgary transit data

1

## The story

Riders are back to 95% of 2019. Feedback per trip is up 12%.

2

## The catch

A 43% jump in complaints that wasn't real.

3

## Ask the data

Plain English in, SQL and the answer out. My proudest part.

Built this week: Airflow · dbt · DuckDB · 27 tests every day · Claude · codychandler.ca/data-demos/calgary-transit

# The hard parts were definitions and people

PEOPLE

## Nobody owned freshness.

The contract named owners, targets and incident tiers.

ANALYTICAL

## The wrong table, or a changed definition, quietly misleads.

One grain per table. The Lost Property fix.

TECHNICAL

## Many shapes of the same thing.

Every client at StellarAlgo; four department codes at the City.

# What I'd build next

## PRIME for clients

Take the internal CS bot to clients as Stellar Insights.

## On-time performance

Add the City's real-time feeds to measure reliability by route.

## Owner sign-off

Agree targets and definitions with the people who run the service.

**The same work at the City: find where numbers disagree, agree one definition with the owners, and make it self-serve.**

# Thank you. Questions?

| | |
|---|---|
| Live demo | **codychandler.ca/data-demos/calgary-transit** |
| Code | **github.com/frontier-platforms/cody-portfolio** |